Universidad Tecnológica del Perú

DISEÑO DE PATRONES

DOCENTE:
Mg. Iván Robles Fernández

MOTIVACIÓN: Trabalenguas

El Rey de Constantinopla está constantinoplizado

Consta que Constanza, no lo pudo desconstantinoplizar

El desconstantinoplizador que desconstantinoplizare al

Rey de Constantinopla, buen desconstantinoplizador será.

¿Algún voluntario que quiera leer el trabalenguas?

🕐

Universidad Tecnológica del Perú

CONOCIMIENTOS PREVIOS

¿Qué entiendes por principios de de Inversión de Dependenica (DIP)?

LOGRO DE LA SESIÓN

Al finalizar la unidad el participante identifica los principios de diseño de los patrones de software para la solución de problemas

IMPORTANCIA DE LA SESIÓN

La aplicación de los patrones de software a permitirá solucionar los problemas habituales de software planteados.

TEMA :

- Principio de Inversión de Dependencia (DIP)
- Principio de Segregación de Interfaces (ISP)

Principio de Inversión de Dependencia (DIP)

Es uno de los cinco principios SOLID de la programación orientada a objetos. Este principio tiene como objetivo reducir la dependencia de clases de alto nivel respecto a clases de bajo nivel, promoviendo así un código más flexible y fácil de mantener.

Establece dos reglas principales:

1. Los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones.
2. Las abstracciones no deben depender de los detalles. Los detalles deben depender de abstracciones.

Principio de Inversión de Dependencia (DIP)

En términos más simples, esto significa que las clases que realizan operaciones importantes (alto nivel) no deben estar acopladas directamente a clases que realizan tareas específicas (bajo nivel).

En su lugar, ambas deberían depender de interfaces o abstracciones.

Principio de Inversión de Dependencia (DIP)

Importancia

- **Flexibilidad:** Permite cambiar las implementaciones de bajo nivel sin afectar a las de alto nivel.
- **Mantenibilidad:** Facilita la modificación del código y la corrección de errores.
- **Testabilidad:** Hace que sea más sencillo realizar pruebas unitarias al permitir el uso de simulaciones (mocks).

Principio de Inversión de Dependencia (DIP) Ventaja

1. **Aumenta la Flexibilidad:** Al depender de abstracciones (interfaces), puedes cambiar las implementaciones concretas sin modificar el código que utiliza estas abstracciones.

   Ejemplo: Si NotificationService usa una interfaz MessageService, puedes cambiar de EmailService a SMSService sin modificar NotificationService.

Principio de Inversión de Dependencia (DIP) Ventaja

2. **Mejora la Mantenibilidad:** Al reducir el acoplamiento entre clases, los cambios en una clase no afectan directamente a otras clases.

Ejemplo: Si necesitas actualizar la lógica de EmailService, no tienes que tocar el código de NotificationService, siempre y cuando EmailService siga cumpliendo con la interfaz MessageService.

Principio de Inversión de Dependencia (DIP) Ventaja

3. **Facilita la Pruebas:** Es más sencillo realizar pruebas unitarias porque puedes usar simulaciones (mocks) de las interfaces.

Ejemplo: Puedes probar NotificationService usando un mock de MessageService, sin necesidad de enviar correos electrónicos reales.

Principio de Inversión de Dependencia (DIP) Ventaja

4. **Promueve el Diseño Modular:** Fomenta la creación de módulos independientes que pueden ser desarrollados y mantenidos por separado.

   Ejemplo: Los equipos pueden trabajar en diferentes implementaciones de MessageService sin interferir en el desarrollo de NotificationService.

Principio de Inversión de Dependencia (DIP) Desventaja

1. **Aumento de la Complejidad Inicial:** La introducción de abstracciones (interfaces) puede hacer que el diseño inicial sea más complejo.

Ejemplo: Definir interfaces como MessageService y asegurarse de que las implementaciones como EmailService y SMSService las cumplan requiere tiempo y esfuerzo adicional.

Principio de Inversión de Dependencia (DIP) Desventaja

2. **Sobrecarga en Pequeños Proyectos:** Para proyectos pequeños, el uso de muchas abstracciones puede ser innecesario y sobrecargar el diseño.

Ejemplo: En una pequeña aplicación de consola, la introducción de interfaces para cada servicio puede resultar excesiva.

Principio de Inversión de Dependencia (DIP) Desventaja

3. **Requiere Buen Conocimiento de Diseño:** Los desarrolladores deben tener un buen entendimiento de los principios SOLID y de diseño orientado a objetos para aplicar correctamente el DIP.

   Ejemplo: Los nuevos desarrolladores pueden tener dificultades para entender cuándo y cómo aplicar el DIP adecuadamente.

Principio de Inversión de Dependencia (DIP) Desventaja

4. **Potencial de Sobre-Ingeniería:** Existe el riesgo de introducir abstracciones innecesarias, lo que puede complicar el código sin un beneficio claro.

   Ejemplo: Crear una interfaz para un servicio que nunca cambiará de implementación puede ser un esfuerzo desperdiciado.

Principio de Inversión de Dependencia (DIP)

Ejemplo: Supongamos que tenemos una aplicación que envía notificaciones a los usuarios. La clase de alto nivel NotificationService depende de una clase de bajo nivel EmailService para enviar correos electrónicos.

```
class EmailService {
    public void sendEmail(String message) {
        // Código para enviar un correo electrónico
    }
}

class NotificationService {
    private EmailService emailService;

    public NotificationService() {
        this.emailService = new EmailService();
    }

    public void sendNotification(String message) {
        emailService.sendEmail(message);
    }
}
```

SIN DIP

En este caso, NotificationService depende directamente de EmailService. Si queremos cambiar la forma en que enviamos notificaciones (por ejemplo, usando SMS en lugar de correo electrónico), tendríamos que modificar NotificationService.

Principio de Inversión de Dependencia (DIP)

Solución:

```
interface MessageService {
    void sendMessage(String message);
}

class EmailService implements MessageService {
    public void sendMessage(String message) {
        // Código para enviar un correo electrónico
    }
}

class SMSService implements MessageService {
    public void sendMessage(String message) {
        // Código para enviar un SMS
    }
}
```

CON DIP

```
class NotificationService {
    private MessageService messageService;

    public NotificationService(MessageService
messageService) {
        this.messageService = messageService;
    }

    public void sendNotification(String message) {
        messageService.sendMessage(message);
    }
}
```

Ahora, NotificationService depende de la interfaz MessageService en lugar de una implementación concreta. Esto permite que NotificationService sea más flexible y fácil de mantener.

Principio de Inversión de Dependencia (DIP)

1. **Caso Práctico 1: Sistema de Envío de Notificaciones**

   Contexto: Tienes que desarrollar un sistema que envía notificaciones. El sistema debe ser capaz de enviar notificaciones por diferentes canales, como correo electrónico o mensajes SMS.

   Objetivo: Aplicar el DIP para que el sistema de notificaciones sea flexible y extensible.

   Instrucciones:
   - Define la interfaz Notifier.
   - Implementa dos clases concretas EmailNotifier y SMSNotifier.
   - Crea la clase NotificationService que depende de la interfaz Notifier.
   - Prueba la implementación.

Principio de Inversión de Dependencia (DIP)

1. **Caso Práctico 2: Sistema de Almacenamiento de Datos**

   Contexto: Desarrolla un sistema para almacenar datos. El sistema debe ser capaz de almacenar datos en diferentes lugares, como archivos o bases de datos.

   Objetivo: Aplicar el DIP para hacer que el sistema de almacenamiento sea flexible y fácil de cambiar.

   Instrucciones:
   - Define la interfaz DataStorage.
   - Implementa dos clases concretas FileStorage y databaseStorege.
   - Crea la clase DataService que depende de la interfaz DataStorage.
   - Prueba la implementación.

Principio de Segregación de Interfaces (ISP)

Establece que un cliente no debe verse forzado a depender de interfaces que no utiliza. En otras palabras, las interfaces deben ser específicas y centradas en un único propósito, para que las clases implementen solo los métodos que realmente necesitan.

Principio de Segregación de Interfaces (ISP) Importancia

- **Evita el Acoplamiento:** Minimiza el acoplamiento entre clases, lo que significa que las clases no dependen de métodos que no utilizan.
- **Facilita Cambios:** Permite hacer cambios en una parte del sistema sin afectar otras partes que no están relacionadas.
- **Mejora la Comprensión:** Hace que el código sea más fácil de entender porque cada interfaz tiene un propósito claro y limitado.

Principio de Segregación de Interfaces (ISP) Ventajas

1. **Flexibilidad:** Las interfaces pequeñas y específicas permiten que las clases implementen solo lo que necesitan.

   Beneficio: Esto facilita la adición de nuevas funcionalidades y la modificación de las existentes sin afectar a otras partes del sistema.

Principio de Segregación de Interfaces (ISP) Ventajas

2. **Mantenibilidad:** Interfaces especializadas hacen que el código sea más fácil de mantener.

   Beneficio: Cambios en una interfaz no afectan a las clases que no la utilizan, reduciendo el riesgo de errores y facilitando las actualizaciones.

Principio de Segregación de Interfaces (ISP) Ventajas

3. **Claridad:** Las interfaces que tienen un propósito claro y específico son más fáciles de entender.

   Beneficio: Facilita a los desarrolladores comprender qué hace cada parte del sistema y cómo encaja en el todo

Principio de Segregación de Interfaces (ISP) Desventajas

1. **Complejidad Inicial:** Diseñar muchas interfaces pequeñas puede hacer que el sistema parezca más complejo al principio.

   Problema: Los desarrolladores pueden necesitar un tiempo para aprender cómo interactúan las múltiples interfaces y cómo usarlas correctamente.

Principio de Segregación de Interfaces (ISP) Desventajas

2. **Número de Interfaces:** Puede llevar a una gran cantidad de interfaces si no se gestiona adecuadamente.

   Problema: Un número elevado de interfaces puede dificultar la navegación y el mantenimiento del sistema si no se tiene una estrategia clara para organizarlas.

Principio de Segregación de Interfaces (ISP) Desventajas

3. **Sobreabundancia de Interfaces:** Puede haber una tendencia a crear muchas interfaces para casos específicos.

   Problema: Esto puede llevar a una sobreabundancia que, en lugar de ayudar, puede complicar la estructura del código.

Principio de Segregación de Interfaces (ISP)

Ejemplo: Imagina que tienes una interfaz Machine con métodos para impresoras y escáneres:

```
interface Machine {
    void print();
    void scan();
    void fax();
}
```

Problema: Si tienes una impresora que solo necesita imprimir y un escáner que solo necesita escanear, ambos se verán obligados a implementar métodos que no usan.

Principio de Segregación de Interfaces (ISP)

Solución:

1. Divide la interfaz en interfaces más específicas:

```
interface Printer {
    void print();
}

interface Scanner {
    void scan();
}

interface FaxMachine {
    void fax();
}
```

Principio de Segregación de Interfaces (ISP)

Solución:

2. Ahora, las impresoras y escáneres implementan solo las interfaces que necesitan:

```
class InkjetPrinter implements Printer {
    public void print() {
        System.out.println("Imprimiendo documento...");
    }
}

class DocumentScanner implements Scanner {
    public void scan() {
        System.out.println("Escaneando documento...");
    }
}
```

Principio de Segregación de Interfaces (ISP)

1. **Caso Práctico 1: Sistema de Multimedia**

   Contexto: Desarrolla un sistema para manejar dispositivos multimedia. Algunos dispositivos solo reproducen audio, otros solo reproducen video, y otros manejan ambos..

   Objetivo: Aplicar el ISP para crear interfaces específicas para cada tipo de dispositivo.

   Instrucciones:
   - Define las interfaces específicas.
   - Implementa las clases correspondientes.
   - Prueba la implementación.

Principio de Segregación de Interfaces (ISP)

1. **Caso Práctico 2: Sistema de Pago**

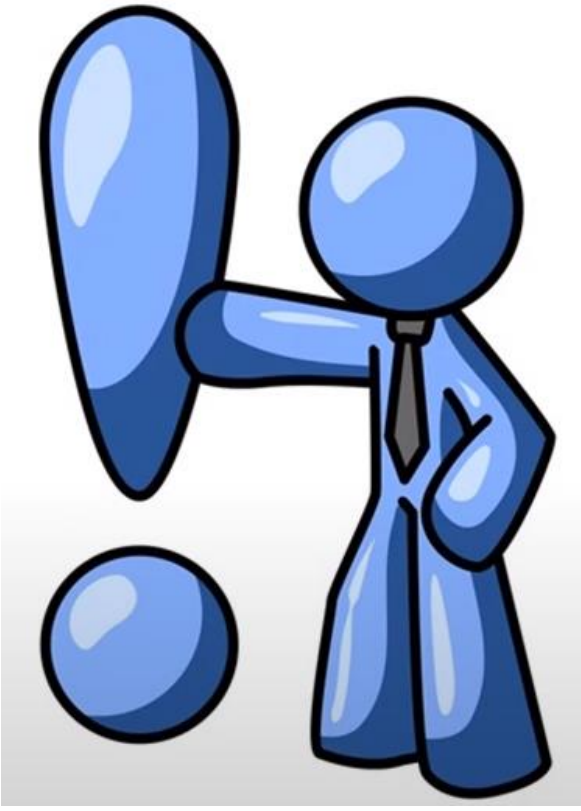
   Contexto: Desarrolla un sistema de pago que maneje diferentes métodos de pago. Algunos métodos procesan pagos en línea, otros en tiendas físicas**.**

   Objetivo: Aplicar el ISP para definir interfaces específicas para diferentes tipos de procesamiento de pagos.

   Instrucciones:
   - Define las interfaces específicas.
   - Implementa las clases correspondientes.
   - Prueba la implementación.

CONCLUSIONES

- **Principio de Inversión de Dependencia (DIP):** El DIP dice que las clases deben depender de abstracciones (interfaces) y no de clases concretas. Las abstracciones deben definir la funcionalidad y las implementaciones concretas deben ajustarse a esas abstracciones. Esto mejora la flexibilidad y facilita los cambios en el código.
- **Principio de Segregación de Interfaces (ISP):** El ISP establece que las interfaces deben ser específicas y pequeñas, adaptadas a las necesidades de los clientes. Las clases no deben implementar métodos que no necesitan. Esto hace el código más claro y manejable.



Universidad Tecnológica del Perú

FACULTADAD DE INGENIERÍA

METACOGNICIÓN

1 ¿ QUÉ HE APRENDIDO ?

SER CONSCIENTE

2 ¿ CÓMO LO HE APRENDIDO ?

DESCRIBIR

3 ¿ QUÉ HA RESULTADO MÁS FÁCIL, MÁS DIFICIL O MÁS NOVEDOSO ?

EVALUAR

4 ¿ CÓMO LO PUEDO MEJORAR? ¿PARA QUÉ ME HA SERVIDO?

PLANIFICAR

Universidad Tecnológica del Perú

FACULTADAD DE INGENIERÍA

¡GRACIAS!

EN LA UTP SABEMOS QUE TODO ES POSIBLE